ServiceNow–Armis Competitive Risk Assessment

Programmatic Risk Heatmap Analysis

Course: ALY 6130 – Enterprise Risk Analytics
Assignment: Module 3 Risk Assessment – Heatmap Development
Company Scenario: ServiceNow Acquisition of Armis
Prepared by: Group 4

Overview

This notebook programmatically visualizes the enterprise risk landscape for ServiceNow's strategic acquisition of Armis. The analysis complements the Excel-based qualitative risk heatmap by generating a programmatic visualization using Python.

The objective is to:

- Translate the risk register into a programmatic dataset
- Compute risk scores using likelihood and impact values
- Classify risks into **High, Medium, and Low categories**
- Generate a visual heatmap-style scatter plot that maps risks according to their likelihood and impact levels.

The values used reflect the same scoring framework defined in the **risk calculation sheet**:

Risk Score = Likelihood × Impact

Risk severity thresholds:

Risk Level	Score Range
High	≥ 45
Medium	20 – 44
Low	< 20

This programmatic visualization ensures consistency between the risk register, calculation sheet, and the qualitative Excel heatmap.

```
# Import required libraries for data handling and visualization

import pandas as pd
import matplotlib.pyplot as plt
```

Import Required Libraries

The first step is to import the Python libraries required for the analysis.

- **pandas** is used to structure and manage the risk dataset.
- **matplotlib** is used to generate the visualization of the risk heatmap.

These libraries are commonly used for data analysis and visualization in risk analytics and business intelligence workflows.

```
from google.colab import files
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
```

```
<IPython.core.display.HTML object>
```

```
Saving ALY.6130_Group4_Module3_Risk_Assessment_updated.xlsx to
ALY.6130_Group4_Module3_Risk_Assessment_updated.xlsx
```

The Excel workbook containing the completed risk register is uploaded into the notebook so it can be accessed programmatically. The uploaded file name is captured dynamically to ensure the dataset can be loaded correctly regardless of how the file is saved in the environment.

```
# Load the RiskRegister sheet from the Excel workbook
df = pd.read_excel(file_name, sheet_name="RiskRegister")

# Display the first rows to verify
df.head()
```


```
{"type":"dataframe","variable_name":"df"}
```


The RiskRegister sheet is loaded from the uploaded Excel workbook using pandas. This step imports the structured risk data, including likelihood scores, impact scores, and priority levels, which will be used to generate the risk heatmap. The first few rows are displayed to verify that the data has been loaded correctly.

```
# Keep only the columns needed for the heatmap
heatmap_df = df[["Risk #", "Likelihood Score", "Impact Score", "Risk
Score", "Priority (H, M, L)"]]

# Display cleaned dataset
heatmap_df.head()
```


```
{"summary":"{\n  \"name\": \"heatmap_df\",\n  \"rows\": 25,\n
\"fields\": [\n  {\n  \"column\": \"Risk #\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
7,\n  \"min\": 1,\n  \"max\": 25,\n
\"num_unique_values\": 25,\n  \"samples\": [\n  9,\n
17,\n  1\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Likelihood Score\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 1,\n  \"min\": 3,\n
\"max\": 9,\n  \"num_unique_values\": 4,\n  \"samples\":
```

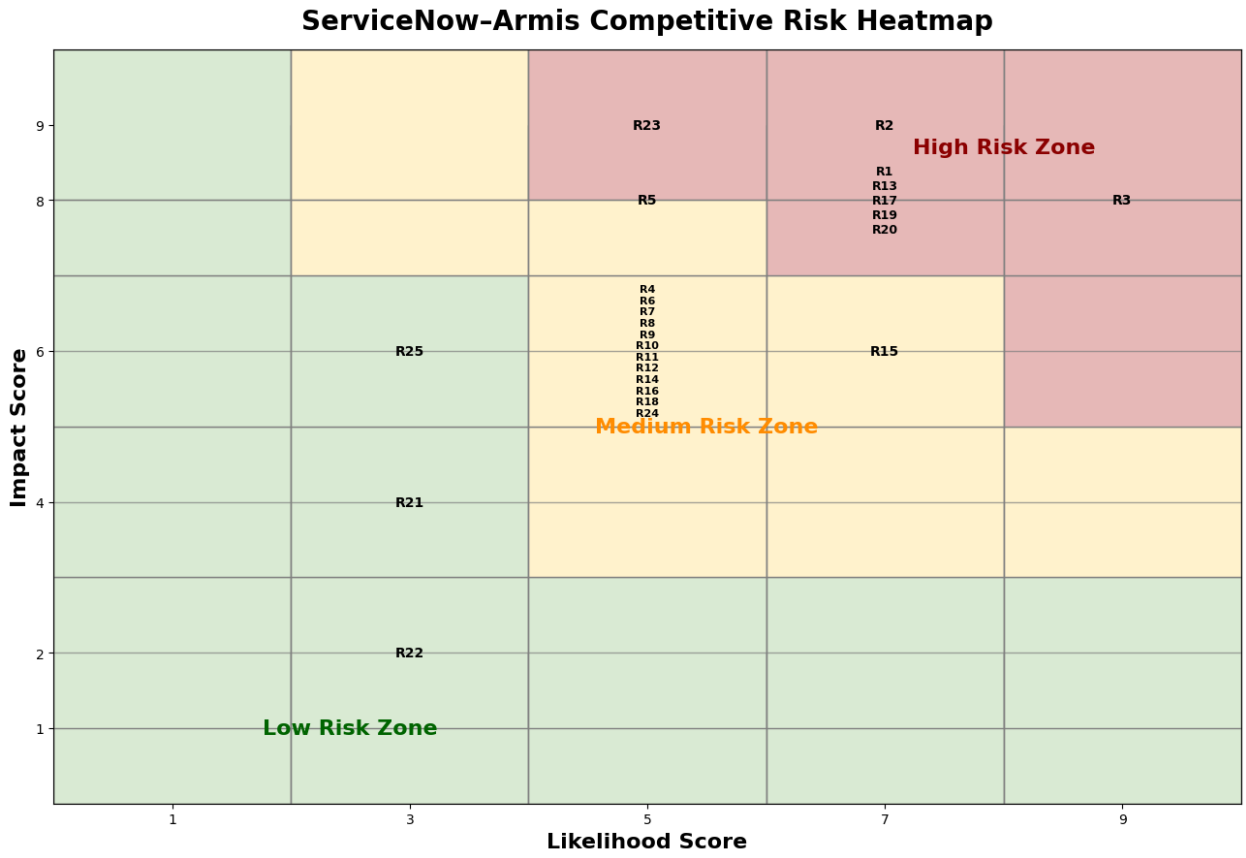
[\n   9,\n   3,\n   7\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Impact Score\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
1,\n   \"min\": 2,\n   \"max\": 9,\n
\"num_unique_values\": 5,\n   \"samples\": [\n   9,\n
2,\n   6\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Risk Score\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 16,\n   \"min\": 6,\n
\"max\": 72,\n   \"num_unique_values\": 10,\n   \"samples\":
[\n   45,\n   63,\n   42\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Priority (H, M, L)\",\n
\"properties\": {\n   \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n   \"samples\": [\n   \"H\",\n
\"M\",\n   \"L\"\n   ],\n   \"semantic_type\": \"\",\
n   \"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"heatmap_df"}

From the full risk register, only the columns required for the heatmap are selected: Risk number, Likelihood Score, Impact Score, Risk Score, and Priority level. This creates a simplified dataset that focuses on the key variables needed to visualize how risks are positioned within the likelihood–impact matrix.

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# ------------------------------
# Allowed scale values
# ------------------------------
likelihood_levels = [1, 3, 5, 7, 9]
impact_levels = [1, 2, 4, 6, 8, 9]

# ------------------------------
# Group risks by (Likelihood, Impact)
# ------------------------------
cell_map = {}

for _, row in heatmap_df.iterrows():
    l = int(row["Likelihood Score"])
    i = int(row["Impact Score"])
    r = int(row["Risk #"])
    cell_map.setdefault((l, i), []).append(r)

for key in cell_map:
    cell_map[key] = sorted(cell_map[key])

# ------------------------------
```

```python
# Figure
# ------------------------------
fig, ax = plt.subplots(figsize=(13, 9))

# ------------------------------
# Draw full heatmap blocks (no gaps)
# ------------------------------
for l in likelihood_levels:
    for i in impact_levels:
        score = l * i

        if score >= 45:
            color = "#E6B8B7"   # High
        elif score >= 20:
            color = "#FFF2CC"   # Medium
        else:
            color = "#D9EAD3"   # Low

        rect = Rectangle(
            (l - 1, i - 1),   # left, bottom
            2,                # width
            2,                # height
            facecolor=color,
            edgecolor="gray",
            linewidth=1
        )
        ax.add_patch(rect)

# ------------------------------
# Add risk labels directly in cells
# ------------------------------
for (l, i), risks in cell_map.items():
    # stacked labels for crowded cells
    label = "\n".join([f"R{r}" for r in risks])

    if len(risks) >= 10:
        fs = 8
    elif len(risks) >= 5:
        fs = 9
    else:
        fs = 10

    ax.text(
        l,
        i,
        label,
        ha="center",
        va="center",
        fontsize=fs,
        fontweight="bold",
```

```
        color="black"
    )

# ----------------------------
# Axis formatting
# ----------------------------
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)

ax.set_xticks(likelihood_levels)
ax.set_yticks(impact_levels)

ax.set_xlabel("Likelihood Score", fontsize=16, fontweight="bold")
ax.set_ylabel("Impact Score", fontsize=16, fontweight="bold")
ax.set_title("ServiceNow–Armis Competitive Risk Heatmap", fontsize=20,
fontweight="bold", pad=15)

# ----------------------------
# Grid lines matching the blocks
# ----------------------------
for x in [0, 2, 4, 6, 8, 10]:
    ax.axvline(x=x, color="gray", linewidth=1, alpha=0.7)

for y in [0, 2, 4, 6, 8, 10]:
    ax.axhline(y=y, color="gray", linewidth=1, alpha=0.7)

# ----------------------------
# Zone labels
# ----------------------------
ax.text(2.5, 1.0, "Low Risk Zone",
        fontsize=16, fontweight="bold", color="darkgreen",
        ha="center", va="center")

ax.text(5.5, 5.0, "Medium Risk Zone",
        fontsize=16, fontweight="bold", color="darkorange",
        ha="center", va="center")

ax.text(8.0, 8.7, "High Risk Zone",
        fontsize=16, fontweight="bold", color="darkred",
        ha="center", va="center")

plt.tight_layout()
plt.show()
```



The following code generates a programmatic risk heatmap using the likelihood and impact scores from the risk register. Each risk is plotted on a two-dimensional matrix where the x-axis represents likelihood and the y-axis represents impact. Color-coded markers indicate the priority classification of each risk (high, medium, or low), while shaded background regions highlight the corresponding risk severity zones.

To ensure readability, risks that share identical likelihood and impact values are slightly offset so that individual risk identifiers remain visible. The resulting visualization provides a clear overview of the distribution of risks associated with the ServiceNow–Armis acquisition and helps highlight which risks fall into the high-priority region requiring closer management attention.

Conclusion

The heatmap visualization highlights how the identified risks are distributed across the likelihood–impact matrix. Several risks fall within the high-risk zone, indicating areas that require close monitoring and proactive mitigation strategies. A number of risks also appear in the medium-risk region, suggesting potential competitive pressures that may escalate if not managed effectively. The concentration of risks in the high-likelihood and high-impact region suggests that competitive pressures following the ServiceNow–Armis acquisition could significantly affect market positioning and strategic growth if not actively monitored. Overall, the heatmap provides a structured view of the risk landscape associated with the ServiceNow–Armis competitive environment and supports prioritization of risk management efforts.